# Linear Regression

## CS 115B: Fundamentals of NLP — Foundations

James Pustejovsky

Brandeis University

Spring 2026

# Today's Lecture

1. The Regression Problem
2. The Linear Model
3. Loss Functions
4. Finding the Optimal Weights
5. Gradient Descent
6. From Regression to Classification

# Supervised Learning: Two Flavors

**Supervised learning:** Learn a function from inputs to outputs using labeled examples.

**Regression**

- Output is **continuous**
- Predict a number
- Examples:
  - House price
  - Temperature
  - Stock price
  - Sentence similarity score

**Classification**

- Output is **discrete**
- Predict a category
- Examples:
  - Spam / not spam
  - Positive / negative sentiment
  - Part-of-speech tag
  - Named entity type

**Today: Regression** $\longrightarrow$ **Next: Classification (Perceptron)**

# The Regression Problem

**Given:**

- Training data: $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\}$
- Each $\mathbf{x}_i \in \mathbb{R}^d$ is a feature vector
- Each $y_i \in \mathbb{R}$ is a continuous target value

**Goal:** Learn a function $f : \mathbb{R}^d \to \mathbb{R}$ such that $f(\mathbf{x}) \approx y$

**Example: Predicting house prices**

| Size (sq ft) | Bedrooms | Age (yrs) | Price ($) |
|---|---|---|---|
| 1500 | 3 | 10 | 250,000 |
| 2000 | 4 | 5 | 350,000 |
| 1200 | 2 | 20 | 180,000 |
| ⋮ | ⋮ | ⋮ | ⋮ |

**Question:** What's the price of a 1800 sq ft, 3 bed, 15 year old house?

# Why Start with Regression?

1. **Simplest supervised learning setting**
   - Continuous outputs are easier to work with than discrete
   - Smooth loss functions with nice mathematical properties

2. **Same linear structure as classification**
   - Both use $\mathbf{w}^\top \mathbf{x} + b$
   - Learn the geometry once, apply it twice

3. **Introduces key concepts**
   - Loss functions
   - Optimization (gradient descent)
   - Overfitting and regularization

4. **Has a closed-form solution**
   - We can solve it exactly — good for building intuition
   - Then see why we might prefer iterative methods

# Linear Regression: The Model

**Assumption:** The output is a **linear function** of the inputs.

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = \mathbf{w}^\top \mathbf{x} + b$$

**Parameters to learn:**

- $\mathbf{w} = (w_1, w_2, \ldots, w_d)^\top$ — the **weight vector**
- $b$ — the **bias** (intercept)

**Interpretation:**

- $w_j$ = how much feature $j$ contributes to the prediction
- $w_j > 0$: feature $j$ increases prediction
- $w_j < 0$: feature $j$ decreases prediction
- $b$ = baseline prediction when all features are zero

# Example: House Price Model

$$\hat{\text{price}} = w_1 \cdot \text{size} + w_2 \cdot \text{bedrooms} + w_3 \cdot \text{age} + b$$

**Suppose we learn:**

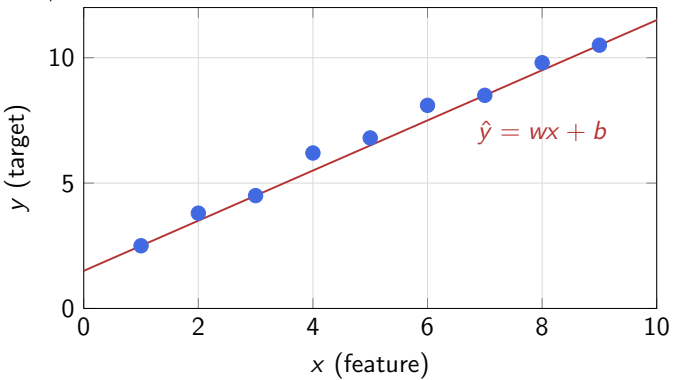
$$\begin{aligned} w_1 &= 150 \quad (\$150 \text{ per sq ft}) \\ w_2 &= 10000 \quad (\$10\text{K per bedroom}) \\ w_3 &= -2000 \quad (\$2\text{K decrease per year of age}) \\ b &= 50000 \quad (\text{baseline}) \end{aligned}$$

**Prediction for (1800 sq ft, 3 bed, 15 yrs):**

$$\begin{aligned} \hat{\text{price}} &= 150 \times 1800 + 10000 \times 3 + (-2000) \times 15 + 50000 \\ &= 270000 + 30000 - 30000 + 50000 \\ &= \$320,000 \end{aligned}$$

# Geometric View: 1D Case

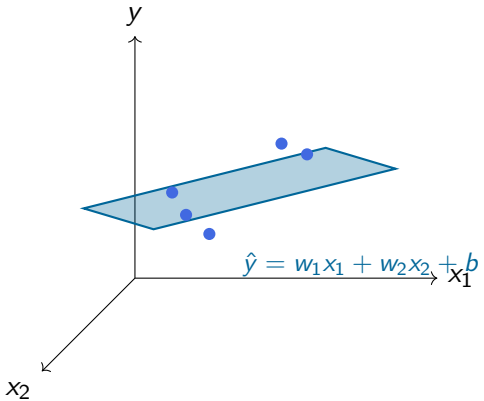
**One feature:** $\hat{y} = wx + b$ is a **line**




**Goal:** Find the line that best fits the data points.

# Geometric View: 2D Case

**Two features:** $\hat{y} = w_1 x_1 + w_2 x_2 + b$ is a **plane**




**In $d$ dimensions:** A **hyperplane** in $(d+1)$-dimensional space.

# The Bias Trick (Again)

**Notational convenience:** Absorb $b$ into $\mathbf{w}$

Append a constant 1 to every input:

$$\tilde{\mathbf{x}} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \\ 1 \end{bmatrix} \qquad \tilde{\mathbf{w}} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \\ b \end{bmatrix}$$

Now: $\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} = \mathbf{w}^\top \mathbf{x} + b$

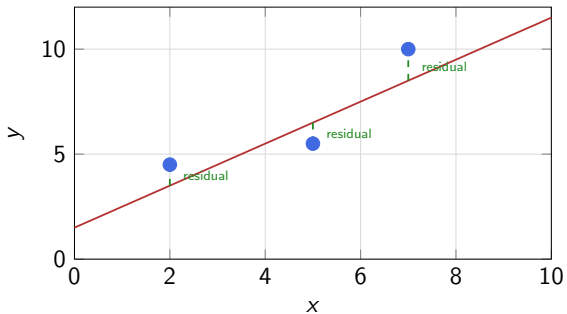
**From now on:** Write $\mathbf{w}^\top \mathbf{x}$ assuming bias is included.

$$\hat{y} = \mathbf{w}^\top \mathbf{x}$$

# How Good Is Our Model?

**Residual:** The error on a single example

$$\text{residual}_i = y_i - \hat{y}_i = y_i - \mathbf{w}^\top \mathbf{x}_i$$




Positive residual: prediction too low. Negative residual: prediction too high.

# Mean Squared Error (MSE)

**Loss function:** Measures overall model quality

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

**Why squared error?**

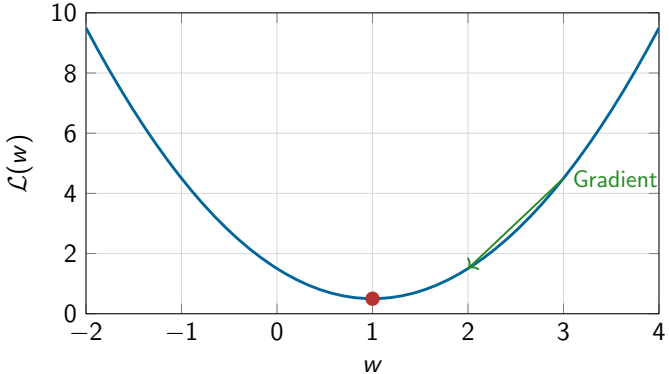
- **Penalizes large errors more:** Error of 10 costs 100, not 10
- **Symmetric:** Over- and under-predictions penalized equally
- **Differentiable:** Smooth function, easy to optimize
- **Convex:** Single global minimum, no local minima traps

**Goal:** Find $\mathbf{w}^*$ that minimizes $\mathcal{L}(\mathbf{w})$

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

# Visualizing the Loss Function

**1D case:** Loss as a function of single weight $w$




**Convexity:** Any downhill path leads to the global minimum.

# Other Loss Functions (Preview)

| Loss | Formula | Properties |
|---|---|---|
| Mean Squared Error | $\frac{1}{n}\sum(y_i - \hat{y}_i)^2$ | Penalizes large errors heavily |
| Mean Absolute Error | $\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$ | Robust to outliers |
| Huber Loss | hybrid | Best of both worlds |

**For classification (coming soon):**

- Zero-one loss (counts errors)
- Hinge loss (perceptron, SVM)
- Cross-entropy loss (logistic regression)

**The choice of loss function shapes what the model learns.**

# Two Approaches to Optimization

**How do we find $\mathbf{w}^*$ that minimizes $\mathcal{L}(\mathbf{w})$?**

**1. Closed-Form Solution**

- Solve $\nabla \mathcal{L} = 0$ analytically
- Get exact answer in one step
- Only works for simple losses
- Can be expensive for large $d$

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

(Normal equations)

**2. Gradient Descent**

- Iteratively improve $\mathbf{w}$
- Take steps downhill
- Works for any differentiable loss
- Scales to large problems

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \mathcal{L}$$

(Iterative updates)

**We'll see both!**

# Closed-Form Solution: Setup

**Matrix notation:** Stack all examples

$$\mathbf{X} = \begin{bmatrix} — & \mathbf{x}_1^\top & — \\ — & \mathbf{x}_2^\top & — \\ & \vdots & \\ — & \mathbf{x}_n^\top & — \end{bmatrix} \in \mathbb{R}^{n \times d} \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n$$

**All predictions at once:**

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} \in \mathbb{R}^n$$

**Loss in matrix form:**

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 = \frac{1}{n}(\mathbf{y} - \mathbf{X}\mathbf{w})^\top(\mathbf{y} - \mathbf{X}\mathbf{w})$$

# Closed-Form Solution: Derivation

**Take the gradient and set to zero:**

$$\begin{aligned}\nabla_{\mathbf{w}}\mathcal{L} &= \nabla_{\mathbf{w}}\left[\frac{1}{n}(\mathbf{y}-\mathbf{X}\mathbf{w})^{\top}(\mathbf{y}-\mathbf{X}\mathbf{w})\right] \\ &= \frac{1}{n}\cdot(-2)\mathbf{X}^{\top}(\mathbf{y}-\mathbf{X}\mathbf{w}) \\ &= \frac{-2}{n}(\mathbf{X}^{\top}\mathbf{y}-\mathbf{X}^{\top}\mathbf{X}\mathbf{w})\end{aligned}$$

**Set $\nabla_{\mathbf{w}}\mathcal{L}=0$:**

$$\mathbf{X}^{\top}\mathbf{X}\mathbf{w}=\mathbf{X}^{\top}\mathbf{y}$$

$$\mathbf{w}^{*}=(\mathbf{X}^{\top}\mathbf{X})^{-1}\mathbf{X}^{\top}\mathbf{y}$$

**The Normal Equations** — exact solution in one matrix operation!

# Closed-Form Solution: Pros and Cons

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

**Pros:**

- Exact solution
- No hyperparameters
- No iteration needed
- Deterministic result

**Cons:**

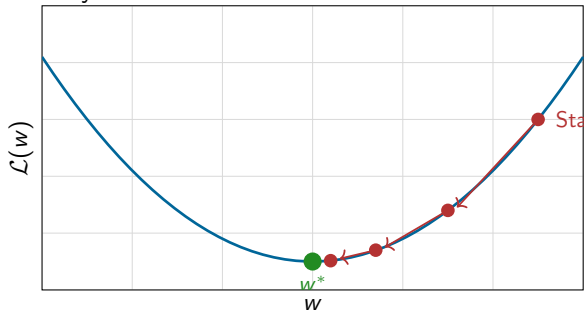
- Must compute $(\mathbf{X}^\top \mathbf{X})^{-1}$
- Cost: $O(nd^2 + d^3)$
- Expensive when $d$ is large
- May be numerically unstable
- Only works for MSE loss

**Rule of thumb:**

- $d < 10,000$: Normal equations often fine
- $d > 10,000$: Use gradient descent

# Gradient Descent: The Idea

**Intuition:** Roll downhill until you reach the bottom.




**Algorithm:** Repeatedly move in the direction of steepest descent.

# Gradient Descent: The Algorithm

**The gradient** $\nabla \mathcal{L}(\mathbf{w})$ points in the direction of steepest *increase*.

**Update rule:** Move in the *opposite* direction:

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \nabla \mathcal{L}(\mathbf{w}^{(t)})$$

**Components:**

- $\mathbf{w}^{(t)}$: Current weights at iteration $t$
- $\nabla \mathcal{L}(\mathbf{w}^{(t)})$: Gradient at current point
- $\eta > 0$: **Learning rate** (step size) — hyperparameter!
- $\mathbf{w}^{(t+1)}$: Updated weights

**Repeat until:** Convergence (gradient $\approx 0$) or max iterations reached.

# Gradient for Linear Regression

**MSE Loss:**

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

**Gradient:**

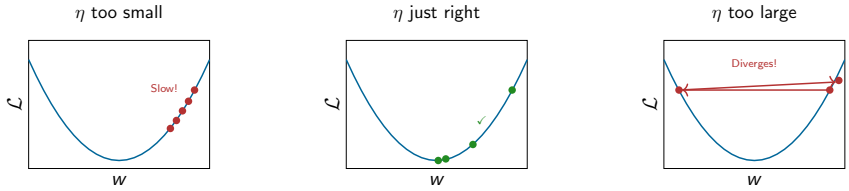
$$\nabla\mathcal{L}(\mathbf{w}) = \frac{-2}{n}\sum_{i=1}^{n}(y_i - \mathbf{w}^\top \mathbf{x}_i)\mathbf{x}_i = \frac{-2}{n}\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w})$$

**Update rule:**

$$\mathbf{w} \leftarrow \mathbf{w} + \frac{2\eta}{n}\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w})$$

**Interpretation:** Adjust weights based on the residuals $(\mathbf{y} - \mathbf{X}\mathbf{w})$, weighted by the features $\mathbf{X}$.

# The Learning Rate $\eta$




**Choosing $\eta$:**

- Too small: Very slow convergence
- Too large: Overshoots, may diverge
- Just right: Fast, stable convergence

**Common strategy:** Start with $\eta = 0.01$ or $0.001$, adjust as needed.

# Stochastic Gradient Descent (SGD)

**Problem:** Computing gradient over all $n$ examples is expensive.

**Solution:** Use a *single random example* (or small batch) per update.

**Batch Gradient Descent:**

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\mathbf{w})$$

**Stochastic Gradient Descent (SGD):**

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla \ell_i(\mathbf{w}) \quad \text{(random } i\text{)}$$

**SGD advantages:**

- Much faster per iteration
- Can escape shallow local minima (noise helps!)
- Scales to massive datasets

**SGD is the workhorse of modern machine learning.**

# What We've Learned

**Linear Regression:**

- Model: $\hat{y} = \mathbf{w}^\top \mathbf{x}$ (linear in features)
- Loss: Mean Squared Error
- Optimization: Normal equations or gradient descent

**Key concepts that transfer to classification:**

- Linear combination of features: $\mathbf{w}^\top \mathbf{x}$
- Loss functions measure model quality
- Gradient descent for optimization
- The bias trick

# But What About Classification?

**Regression:** Predict continuous $y \in \mathbb{R}$

$$\hat{y} = \mathbf{w}^\top \mathbf{x} \quad \text{(any real number)}$$

**Classification:** Predict discrete $y \in \{-1, +1\}$ or $\{0, 1\}$

**Problem:** $\mathbf{w}^\top \mathbf{x}$ gives a real number, not a class!

**Solutions:**

1. **Perceptron:** Apply $\text{sign}(\cdot)$ to get $\pm 1$

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x})$$

2. **Logistic Regression:** Apply sigmoid to get probability

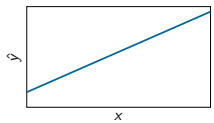
$$P(y = 1) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

# Preview: The Perceptron

**Same linear structure, different output:**

**Linear Regression**

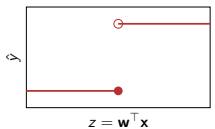
$$\hat{y} = \mathbf{w}^\top \mathbf{x}$$




Continuous output

**Perceptron**

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x})$$




Binary output: $\pm 1$

**Next lecture: The Perceptron Algorithm**

# Summary

**Linear Regression:**

- Predicts continuous values using $\hat{y} = \mathbf{w}^\top \mathbf{x}$
- Learns weights by minimizing squared error
- Can solve exactly (normal equations) or iteratively (gradient descent)

**Key Takeaways:**

- Linear models: simple, interpretable, foundational
- Loss functions quantify what "good" means
- Gradient descent is general-purpose optimization
- Same framework extends to classification

**Coming up:**

- Perceptron: Linear classification with hard decisions
- Logistic Regression: Linear classification with probabilities
- Multi-layer networks: Non-linear models

# Next: The Perceptron Algorithm